Beyond the Syllabus

Union-Find and Graph Connectivity: Provable Limits, Concurrent Algorithms, and Planet-Scale Systems

Week 3 Supplement – TOAE209/TOAH209: Design and Analysis of Algorithms

Foreign Trade University

Outline

Why look beyond the syllabus?

- This week you proved that Union-Find, with union by rank and path compression, costs $O(m\,\alpha(n))$ total time for m operations – essentially constant per operation. You also traced BFS and DFS, both $O(n+m)$.
- Four questions a curious student should ask next – each one turns out to be an active (or very recently settled) research question:
  1. Is $O(m\,\alpha(n))$ actually the **best possible**, or could a smarter data structure beat it?
  2. Union-Find only handles edges being **added**. What if edges can also be **deleted**?
  3. What happens if **many threads** call `union` at the same time?
  4. Your BFS/DFS ran on a graph with a few hundred vertices. What does it take to run the **same idea** on a graph with **128 billion edges**?

How this supplement is different from a survey

Every section below walks through the *actual mechanics* of an algorithm – with a hand-traceable running example – not just a headline result. Where I quote a specific bound or number, it is sourced to a specific paper (cited on the slide).

Recap: the bound you proved this week

Theorem (this week's lecture notes)

With union by rank *and* path compression, a sequence of m `find`/`union` operations on n elements takes $O(m\,\alpha(n))$ total time, where α is the inverse Ackermann function.

- This is an **upper bound**: a guarantee that this particular algorithm (rank + compression) never does worse.
- It says nothing about whether some entirely different data structure – with a cleverer invariant, more bits of state per element, anything – could do *better*. Ruling that out requires a completely different kind of argument: a **lower bound** that applies to *every* possible algorithm.
- Fredman and Saks (STOC 1989, "The Cell Probe Complexity of Dynamic Data Structures") proved exactly such a lower bound, 14 years after Tarjan's 1975 upper bound. *DOI:* `10.1145/73007.73040`.

Just how slowly does $\alpha(n)$ actually grow?

Before the lower bound, get a feel for the function being bounded. Recall $\log^* n$ ("iterated log"): the number of times you apply $\log_2$ before the result drops to ≤ 1. $\alpha(n)$ grows even *slower* than $\log^* n$.

n	step 1	step 2	step 3	step 4	$\log^* n$
2	1	–	–	–	1
4	2	1	–	–	2
16	4	2	1	–	3
65,536	16	4	2	1	4
2^{65536} (a 19,729-digit number)	65,536	16	4	$2 \to 1$	5

Hand-checkable

$\log_2 65536 = 16$, $\log_2 16 = 4$, $\log_2 4 = 2$, $\log_2 2 = 1$: four steps, so $\log^*(65536) = 4$. And 2^{65536} has about 19,729 decimal digits – far more than the $\sim 10^{80}$ atoms in the observable universe – yet needs only *one more* step: $\log^*(2^{65536}) = 5$. No n you could ever write down needs $\log^* n > 5$, and $\alpha(n)$ is even flatter than this.

The lower bound: proof idea (Fredman–Saks, 1989)

Model: the *cell-probe* model – charge an algorithm only for memory *words read or written*, ignoring all local computation (the most generous possible model for an algorithm to work in, so a lower bound here is very strong).

- **Adversary construction.** Build a specific, structured sequence of m interleaved `union`/`find` calls on n elements, organized into **epochs**, where each epoch forces some `find` to depend on information written many epochs earlier.
- **Counting argument.** Few memory probes per `find` means little information can flow between far-apart epochs – but the adversary sequence needs a specific number of "surviving" epochs to avoid contradiction, and that number grows like the inverse Ackermann function.
- **Conclusion:** *any* algorithm (not just rank + compression) must spend $\Omega(m\,\alpha(n))$ total cell probes on some such sequence.

Why only a sketch

The full "chronogram technique" proof is one of the most technically involved lower bounds in data structures – normally only sketched even in graduate courses. Take away *what kind of argument* proves a lower bound (adversary + information counting), a different skill from proving an upper bound.

The lesson: a 14-year-old gap, closed

Year	Result	What it establishes
1975	Tarjan (upper bound)	$O(m\,\alpha(n))$ suffices
1984	Tarjan & van Leeuwen (refinement)	tight analysis of rank + compression
1989	Fredman & Saks (lower bound)	$\Omega(m\,\alpha(n))$ is *necessary*

The takeaway

The algorithm you hand-traced this week is not a good-enough approximation of some better algorithm nobody has found yet. It is, up to constant factors, **the fastest possible** amortized Union-Find in this model – full stop. That is a rare and satisfying thing to be able to say about an algorithm you can implement in 20 lines of code.

The limitation: Union-Find only grows

- Everything this week assumed edges are only ever **added**. That is exactly the right model for Kruskal's MST (a later week) – but real networks lose edges too: a road closes, a link goes down, a friendship ends.
- **Question:** can we maintain "are u and v connected?" under both edge insertions *and* deletions, still faster than recomputing from scratch?

The naive baseline

After every deletion, rerun BFS/DFS from scratch: $O(n + m)$ per update. For a graph with millions of updates (a live social graph, a router table), this is far too slow – we want something closer to Union-Find's near-constant cost, in both directions.

Why Union-Find itself can't be patched

Union-Find has no memory of *why* two elements got connected – once you union(x, y), the specific edge is forgotten. Deleting "the edge that caused the union" is meaningless to the data structure: there is no way back.

The key idea: give every edge a *level*

Holm, de Lichtenberg, and Thorup (**HDT**, J. ACM 2001) maintain a spanning forest F where every tree edge e carries a level $\ell(e) \in \{0, \dots, L\}$, $L = \lceil \log_2 n \rceil$.

The level invariant

Let $F_i = \{e \in F : \ell(e) \geq i\}$, so $F = F_0 \supseteq F_1 \supseteq \cdots \supseteq F_L$. Invariant: every tree of F_i has **at most $n/2^i$ vertices**. New edges always start at level 0.

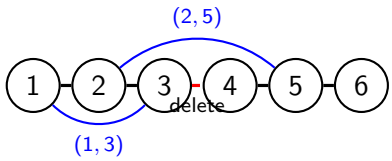
- Each level's forest F_i is stored in an **Euler-tour tree**: a balanced structure supporting `link`, `cut`, `find-root`, and `subtree-size`, all in $O(\log n)$ – and (with extra bookkeeping) it can also report "some non-tree edge touching this subtree" in $O(\log n)$ amortized.
- Non-tree edges are stored per level too, indexed by their endpoints, so we can ask "does any level-i non-tree edge leave this subtree?"

Deleting a tree edge: the replacement-edge search

Algorithm: $\textsc{Delete}(e)$, where e is a tree edge at level $i = \ell(e)$

1. Cut e from the Euler-tour trees at levels $0, 1, \ldots, i$ – this splits one tree into two pieces, call them T_1, T_2.
2. For level $j = i$ down to 0:
   1. Let T_{small} be the smaller of $T_1 \cap F_j$, $T_2 \cap F_j$ (found via an $O(\log n)$ size query).
   2. **Promote** every tree edge inside T_{small} from level j to level $j+1$ (this is safe: T_{small} has $\leq n/2^{j+1}$ vertices).
   3. Scan the non-tree edges touching T_{small} at level j, one at a time. For each edge (x, y):
      - if $y \notin T_{\text{small}}$ (it reaches the *other* piece): **replacement found** – link it as a tree edge at level j. Stop.
      - else ($y \in T_{\text{small}}$, a "dud"): promote it to level $j+1$ too, and keep scanning.
3. If level 0 is exhausted with no replacement: the deletion genuinely increased the number of connected components.

Running example: deleting a bridge edge



solid = tree edges (level 0); blue = non-tree edges (level 0)

Delete tree edge $(3,4)$, level $i = 0$. Splits $\{1,2,3\}$ vs. $\{4,5,6\}$ (tie, either could be T_{small}; take $\{1,2,3\}$).

1. Promote $(1,2)$ and $(2,3)$ to level 1 (still tree edges, now also in F_1; $|\{1,2,3\}| = 3 \leq n/2^1 = 3$ ✓).
2. Scan non-tree edges touching $\{1,2,3\}$: first $(1,3)$ – both endpoints inside T_{small}, a **dud**, promote to level 1.
3. Next $(2,5)$ – endpoint 5 is in the *other* piece: **replacement found**. Link $(2,5)$ as a tree edge at level 0.

Result

New spanning tree: $(1,2)_{L1}, (2,3)_{L1}, (2,5)_{L0}, (4,5)_{L0}, (5,6)_{L0}$ – all six vertices reconnected, and two edges are now "used up" at level 1 where they can help future, smaller-scale searches.

Why this is fast: the amortized argument

- There are only $L = O(\log n)$ levels, and each edge's level only ever **increases** – so any single edge can be promoted at most $O(\log n)$ times over its entire lifetime.
- Charge each promotion to the edge being promoted: total promotion work over the *whole* sequence of m operations is $O(m \log n)$, i.e. $O(\log n)$ **amortized** per operation.
- Each `Delete` also does $O(\log n)$ levels of "search," each costing $O(\log n)$ for the Euler-tour-tree operations – an extra $O(\log^2 n)$ worst-case-looking cost, but the search that finds a replacement (or exhausts a level) is exactly what triggers the promotions that pay for it.
- Net result: $O(\log^2 n)$ **amortized** time per update, $O(\log n / \log \log n)$ per connectivity query, with a refinement – Holm, de Lichtenberg, Thorup, J. ACM 2001.

The race for *worst-case* (not just amortized) bounds

HDT's $O(\log^2 n)$ is *amortized*: one single update could still be slow, as long as the average stays low. Getting the *same* guarantee on **every single** update turned into its own 25-year research program:

Year	Result	Bound
2001	Holm, de Lichtenberg, Thorup	$O(\log^2 n)$ amortized
2013	Kapron, King, Mountjoy (SODA best paper)	polylog *worst-case*, randomized
2017	Nanongkai, Saranurak, Wulff-Nilsen (FOCS)	$n^{o(1)}$ worst-case, Las Vegas
2023–24	Goranci, Henzinger, Nanongkai, Saranurak, Thorup, Wulff-Nilsen	$\tilde{O}(n)$ worst-case, exact *edge connectivity*
2024	Jin, Sun, Thorup (SODA)	dynamic min-cut, subpolynomial

An open question, resolved – this decade

The 2023–24 result answers, in the affirmative, a question Thorup himself posed in *Combinatorica*, 2007: can *exact edge connectivity* (not just yes/no reachability) be maintained with worst-case $o(m)$ update time? As of this month (August 2026), arXiv preprints (e.g. 2510.08297, 2605.15173, 2601.20137) show the bounds are **still being pushed further, right now**.

The problem: Union-Find was never designed for parallel hardware

- Modern CPUs have dozens of cores; GPUs have thousands. Computing connected components of a huge graph means many threads calling `union` and `find` *at the same time*, on the *same* `parent[]` array.
- The sequential algorithm from this week is **not safe** under concurrency – two threads racing on the same memory can corrupt the structure.

A concrete race: a cycle appears

x, y start as singleton roots. Thread A runs `union`(x, y); Thread B runs `union`(y, x), at the same instant.

1. Both threads read $root(x) = x$ and $root(y) = y$ *before either writes*.
2. A decides to set $parent[x] \leftarrow y$; B decides to set $parent[y] \leftarrow x$ – based on stale information.
3. Both writes go through: $parent[x] = y$ **and** $parent[y] = x$. Now `find`(x) loops **forever**. The tree invariant is destroyed.

The fix: compare-and-swap, plus a tie-break rule

CAS-UNION(x, y)

1. **loop:** $r_x \leftarrow \text{FIND}(x)$; $r_y \leftarrow \text{FIND}(y)$
2. if $r_x = r_y$: return (already unioned)
3. if $r_x < r_y$: swap(r_x, r_y) – *fixed, deterministic tie-break*
4. if CAS$(parent[r_x],\ r_x,\ r_y)$ succeeds: return
5. else: **retry** the whole loop (someone else moved r_x concurrently)

FIND(x) – with best-effort path halving

1. while $parent[x] \neq x$: let $p \leftarrow parent[x]$; attempt CAS$(parent[x], p, parent[p])$ (ok if it fails – just an optimization); $x \leftarrow p$
2. return x

Why this fixes the race

Two ideas doing the real work

- The **deterministic tie-break** (*always* attach the smaller root under the larger, regardless of which thread got there first) rules out the two-writer cycle from the previous slide *by construction* – both threads, no matter what they observe, agree on which direction the edge must point.
- The **CAS-and-retry loop** means a losing thread *notices* it lost (its compare-and-swap fails because $parent[r_x]$ no longer matches what it read) and safely recomputes, instead of overwriting blindly with stale information.

Tracing the fix on the exact race that broke us

Same scenario: A runs $\texttt{union}(x, y)$, B runs $\texttt{union}(y, x)$, concurrently. Say $x < y$, so the tie-break always targets $\mathit{parent}[x] \leftarrow y$, no matter which thread computes it.

1. Both A and B read $r_x = x$, $r_y = y$; both apply the *same* tie-break rule and both attempt CAS($\mathit{parent}[x]$, x, y).
2. Say A's CAS runs first: $\mathit{parent}[x]$ was x, so it succeeds – $\mathit{parent}[x] \leftarrow y$. A returns.
3. B's CAS now runs: it expects $\mathit{parent}[x]$ to still be x, but it is now y – **CAS fails**. B retries: recomputes $r_x = \textsc{Find}(x) = y$, $r_y = \textsc{Find}(y) = y$. Now $r_x = r_y$: B returns immediately.

No cycle, no lost update

Exactly one union happens; the loser detects the conflict and safely no-ops. This is the general pattern behind lock-free data structures: never overwrite blindly, always verify-then-commit, and retry on failure.

Jayanti and Tarjan (PODC 2016): making the guarantee rigorous

The CAS loop above is the accessible *idea*. Proving a tight *worst-case* time bound under an adversarial thread scheduler is much harder – solved by Sam Jayanti and **Robert Tarjan** (the same Tarjan behind this week's classical rank-and-compression analysis, four decades earlier).

The result

A deterministic **wait-free** algorithm (double-CAS), plus two randomized algorithms (ordinary CAS only), all achieve total work $O\big(m(\log(\frac{np}{m}+1)+\alpha(n,\frac{m}{np}))\big)$ for n elements, m operations, p concurrent processes – essentially the *same* inverse-Ackermann-type bound as the sequential algorithm, even under adversarial scheduling.

Why it matters in practice

This style of algorithm is now, per the subsequent literature, the practical basis for the **fastest known** connected-components code on both CPUs and GPUs.

From a homework graph to the entire Web

- This week's BFS/DFS/Union-Find are all $O(n + m)$ – instant on a graph with a few hundred vertices. What about the **Hyperlink2012** web graph: 3.5 **billion** vertices, 128 **billion** edges?

Dhulipala, Blelloch, Shun (SPAA 2018) – the GBBS/Ligra system

Computing all connected components of Hyperlink2012 took **25.0 seconds** on a *single* shared-memory machine with 72 cores.

System	Hardware	Time
GBBS/Ligra (2018)	1 machine, 72 cores	**25.0 s**
Distributed cluster (Stergiou et al.)	1000 nodes, 12,000 cores, 128TB RAM	341 s
Supercomputer (Slota et al.)	$>$ 8000 cores	$\approx$ 6 min

13.6$\times$ faster than the cluster, using 166$\times$ fewer cores and 128$\times$ less memory. This work (Ligra/GBBS/Aspen) won the 2022 ACM Paris Kanellakis Theory and Practice Award.

How? The same algorithm, engineered to be work-efficient in parallel

- No new algorithmic idea is needed at the core: a **direction-optimizing BFS** (switch between expanding the frontier "top-down" from visited vertices, and "bottom-up" by checking unvisited vertices for a visited neighbor, whichever is cheaper for the current frontier size) combined with a **parallel-safe Union-Find** – exactly the CAS-based `union` from the previous section, applied to all edges at once.
- Total work stays $O(n + m)$; what changes is that the work is spread across many cores with high memory locality and (nearly) no synchronization overhead – an *engineering* achievement built directly on top of this week's algorithms and last section's concurrency result.

A sharper shortcut for real graphs: sampling (Afforest)

Real graphs (web, social networks) are **power-law**: a handful of hub vertices touch a huge share of all edges, and almost every vertex sits in one giant component. Sutton, Ben-Nun, and Barak (IPDPS 2018, *"Optimizing Parallel Graph Connectivity Computation via Subgraph Sampling"*) exploit exactly this:

Afforest, three steps

1. **Sample rounds:** for 2 rounds, link every vertex to just *one* neighbor (not all of them). On a power-law graph this alone merges almost everything into a few giant components.
2. **Identify the giant component:** randomly sample 1024 vertices; the most frequent component label among them is (almost certainly) the giant component.
3. **Skip and finish:** scan all edges once, but **skip** any edge whose two endpoints are *already* both known to be in the giant component – the vast majority of edges. Union the (few) remaining edges normally.

Same $O(n + m)$ worst case, but in practice most of the m is never touched.

FastCC (2026): one BFS, from the right vertex

- ACM Transactions on Architecture and Code Optimization, 2026 (*DOI:* `10.1145/3803020`) pushes the power-law idea further: instead of random sampling rounds, **fix the BFS root at the single highest-degree vertex**, and run one BFS.
- On a power-law graph, that one BFS from the "hub" typically reaches almost the entire giant component in a single, predictable, well-load-balanced pass – avoiding the irregular frontier sizes that make plain BFS slow and imbalanced on these graphs.

Reported results

10.6–55.5× (average 37.8×) faster than prior state-of-the-art connected-components implementations (including systems built on the GBBS/Afforest ideas above), up to 2.87× less peak memory – and it is the engine behind a top-ranked **GreenGraph500** entry (a supercomputing benchmark ranked by *energy efficiency*, not just raw speed).

The lesson

Same two primitives, all the way up

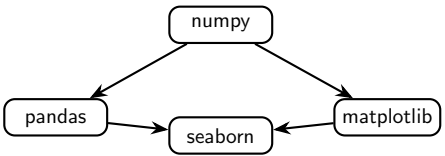
Every system in this section – a homework BFS, a 72-core machine indexing the web, a 2026 energy-efficiency record – is built from exactly the two ideas you hand-traced this week: **graph traversal** and **Union-Find**. What changes from a classroom exercise to a production system is not the algorithm's asymptotic bound, but decades (and counting) of work on making that same $O(n + m)$ bound actually fast on real hardware and real, power-law-shaped data.

Kahn's algorithm, everywhere

No new research needed here – just naming where the exact algorithm from this week's notes (KAHN(*adj*), Algorithm in Section 6) runs **today**:

- `pip`, `npm`, `cargo`: before installing anything, they topologically sort the package dependency DAG.
- **Bazel** / **Make**: build targets are scheduled in dependency order – exactly Kahn's algorithm over the build graph.
- **PyTorch autograd** and **CUDA graph capture**: operations in a computation graph execute only after their inputs are ready – a topological order of the op graph.
- **Airflow** DAGs and agent-orchestration frameworks (e.g. LangGraph): task nodes run only after their declared dependencies finish.

Worked example: resolving a package install order



In-degrees: *numpy*=0, *pandas*=1, *matplotlib*=1, *seaborn*=2.

- Queue ← [*numpy*] (only in-degree-0 vertex).
- Dequeue *numpy*; output it; decrement *pandas* → 0, *matplotlib* → 0; enqueue both.
- Dequeue *pandas*; output it; decrement *seaborn* → 1 (not yet 0).
- Dequeue *matplotlib*; output it; decrement *seaborn* → 0; enqueue it.
- Dequeue *seaborn*; output it.

Result

Install order: *numpy* → *pandas* → *matplotlib* → *seaborn* (or *numpy* → *matplotlib* → *pandas* → *seaborn* – both valid). This is *literally* KAHN(*adj*) from this week's notes, run on a dependency graph instead of a textbook example.

Summary

1. **Provable optimality:** Fredman & Saks (1989) proved Union-Find's $O(m\,\alpha(n))$ bound is not just good – it is the best *any* algorithm can do, in a very general model.
2. **Deletions reopen everything:** allowing edges to disappear turns a 20-line data structure into a 25-year (and still active) research program – from $O(\log^2 n)$ amortized (2001) to worst-case results still being refined in 2026.
3. **Concurrency needs its own proof:** a CAS-loop-with-tie-break fixes the obvious race, but Jayanti & Tarjan (2016) needed a genuine research result to guarantee it under *any* adversarial schedule – now the practical backbone of real CPU/GPU connected-components code.
4. **Scale is an engineering frontier, not a new algorithm:** this week's exact $O(n + m)$ ideas, tuned for real hardware and power-law graphs, index a 128-billion-edge web graph in 25 seconds – and a 2026 paper is still improving it.

Looking ahead

Next week (Kleinberg & Tardos, Chapter 4) reuses BFS as a stepping stone toward **greedy algorithms**, and Union-Find reappears as the core data structure behind Kruskal's Minimum Spanning Tree algorithm.